



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: HERNÁNDEZ FIERROS SONIA

N° de Cuenta: 319194359

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 17 de septiembre del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Se posiciona el centro del rover (X amarilla) en (0, 4, -4). A partir de él se dibuja la cabina (8×4×6 unidades) y la base (10×1.5×8). El centro de la base se guarda en modelaux2, que es el punto de partida de todas las ramas del árbol jerárquico.

```
358
359     model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f)); //NOS POSICIONAMOS EN EL CENTRO
360     modelaux = model; //guardamos la matriz de modelo para que la base se mueva con el objeto
361     // Creando la cabina del rover ¿CUÁNTAS UNIDADES MEDIRÁ EN Z?
362     model = glm::translate(model, glm::vec3(1.0f, 2.0f, 0.0f)); //PARA LLEGAR AL CENTRO DE LA CABINA
363     model = glm::scale(model, glm::vec3(8.0f, 4.0f, 6.0f));
364     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
365     //La línea de proyección solo se manda una vez a menos que en tiempo de ejecución se programe
366     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
367     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
368     color = glm::vec3(1.0f, 0.0f, 1.0f);
369     glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
370     meshList[0] -> RenderMesh(); //dibuja cubo y pirámide triangular
371     //meshList[3] -> RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide
372     //sp.render(); //dibuja esfera
373
374
375     // BASE
376     model = glm::mat4(1.0); //Si dejamos esta línea, la base no estará unida al centro del objeto
377     /*En su lugar usamos la matriz auxiliar modelaux para que la base se mueva con el objeto.
378     Lo que debemos de saber es: de las transformaciones geométricas que se aplican a la cabina y
379     cuales queremos que se apliquen a la base. En este caso, sólo queremos que se aplique la traslación.
380     Por lo tanto, debemos de guardar en modelaux sólo la traslación del origen y luego aplicarla a la base.
381     */
382     model = modelaux;
383     //NOS POSICIONAMOS EN EL CENTRO DE LA BASE
384     model = glm::translate(model, glm::vec3(0.0f, -0.75f, 0.0f));
385     modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
386     model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f));
387     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
388     color = glm::vec3(0.0f, 1.0f, 0.0f);
389     glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
390     meshList[0] -> RenderMesh();
```

Brazo con pinza. Cada articulación gira con el teclado (F, G, H, J, K, L). La pinza son dos pirámides.

```

398 //BASE PARA EL BRAZO
399 //BRAZO
400 model = modelaux2;
401 model = glm::translate(model, glm::vec3(-3.5f, 1.25f, 2.5f));
402 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
403 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
404 color = glm::vec3(0.0f, 0.0f, 1.0f);
405 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
406 meshList[0]->RenderMesh();
407
408 model = glm::translate(model, glm::vec3(0.0f, 0.8f, 0.0f));
409 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
410 modelaux = model;
411 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
412 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
413 color = glm::vec3(0.5f, 0.5f, 0.5f);
414 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
415 sp.render();
416
417 model = modelaux;
418 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //rotar alrededor de z 45 grados
419 model = glm::translate(model, glm::vec3(0.0f, 2.3f, 0.0f));
420 modelaux = model;
421 model = glm::scale(model, glm::vec3(0.5f, 4.0f, 0.5f));
422 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
423 color = glm::vec3(1.0f, 0.0f, 0.0f);
424 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
425 meshList[2]->RenderMeshGeometry();
426
427 model = modelaux;
428 model = glm::translate(model, glm::vec3(0.0f, 2.3f, 0.0f));
429 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
430 modelaux = model;
431 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
432 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
433 color = glm::vec3(0.5f, 0.5f, 0.5f);
434 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos

```

```

434 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
435 sp.render();
436
437 model = modelaux;
438 model = glm::translate(model, glm::vec3(2.3f, 0.0f, 0.0f));
439 modelaux = model;
440 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //rotar alrededor de z -90 grados
441 model = glm::scale(model, glm::vec3(0.5f, 4.0f, 0.5f));
442 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
443 color = glm::vec3(1.0f, 1.0f, 0.0f);
444 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
445 meshList[2]->RenderMeshGeometry();
446
447 model = modelaux;
448 model = glm::translate(model, glm::vec3(2.3f, 0.0f, 0.0f));
449 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
450 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
451 modelaux = model;
452 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
453 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
454 color = glm::vec3(0.5f, 0.5f, 0.5f);
455 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
456 sp.render();
457
458 model = modelaux;
459 model = glm::translate(model, glm::vec3(0.0f, 0.55f, 0.0f));
460 modelaux = model;
461 model = glm::scale(model, glm::vec3(0.9f, 0.3f, 0.6f));
462 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
463 color = glm::vec3(1.0f, 0.5f, 0.0f);
464 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
465 meshList[0]->RenderMesh();

```

```

465 meshList[0]->RenderMesh();
466
467 model = modelaux;
468 model = glm::translate(model, glm::vec3(0.3f, 0.15f, 0.0f));
469 model = glm::rotate(model, glm::radians(15.0f - mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
470 model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
471 model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.5f));
472 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
473 color = glm::vec3(0.0f, 1.0f, 1.0f);
474 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
475 meshList[4]->RenderMeshGeometry();
476
477 model = modelaux;
478 model = glm::translate(model, glm::vec3(-0.3f, 0.15f, 0.0f));
479 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4() - 15.0f), glm::vec3(0.0f, 0.0f, 1.0f));
480 model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
481 model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.5f));
482 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
483 color = glm::vec3(0.0f, 1.0f, 1.0f);
484 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
485 meshList[4]->RenderMeshGeometry();
486
487 model = modelaux2;
488 //LLANTA FRONTAL +Z

```

Llanta frontal con su pata en L y un eje intermedio. Gira con las teclas 1 a 6; las otras 5 llantas son iguales.

```

488 //LLANTA FRONTAL +Z
489 model = glm::translate(model, glm::vec3(-4.0f, 0.75f, 4.3f));
490 modelaux = model;
491 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
492 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
493 color = glm::vec3(0.5f, 0.5f, 0.5f);
494 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
495 sp.render();
496
497 model = modelaux;
498 model = glm::translate(model, glm::vec3(-1.5f, -0.5f, 0.0f));
499 model = glm::rotate(model, glm::radians(18.43f), glm::vec3(0.0f, 0.0f, 1.0f));
500 model = glm::scale(model, glm::vec3(3.16f, 0.4f, 0.5f));
501 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
502 color = glm::vec3(1.0f, 0.5f, 0.0f);
503 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
504 meshList[0]->RenderMesh();

```

```

505
506     model = modelaux;
507     model = glm::translate(model, glm::vec3(-3.0f, -1.0f, 0.0f));
508     modelaux = model;
509     model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
510     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
511     color = glm::vec3(0.5f, 0.5f, 0.5f);
512     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
513     sp.render();
514
515     model = modelaux;
516     model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.5f));
517     modelaux = model;
518     model = glm::translate(model, glm::vec3(-0.375f, 0.0f, 0.0f));
519     model = glm::scale(model, glm::vec3(1.25f, 0.5f, 0.5f));
520     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
521     color = glm::vec3(1.0f, 1.0f, 1.0f);
522     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
523     meshList[0]->RenderMesh();
524     model = modelaux;
525     model = glm::translate(model, glm::vec3(-0.75f, -0.875f, 0.0f));
526     model = glm::scale(model, glm::vec3(0.5f, 2.25f, 0.5f));
527     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
528     color = glm::vec3(1.0f, 1.0f, 1.0f);
529     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
530     meshList[0]->RenderMesh();
531
532     model = modelaux;
533     model = glm::translate(model, glm::vec3(-0.75f, -1.75f, 0.45f));
534     modelaux = model;
535     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
536     model = glm::scale(model, glm::vec3(0.25f, 0.9f, 0.25f));
537     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
538     color = glm::vec3(0.5f, 0.5f, 0.5f);
539     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
540     meshList[2]->RenderMeshGeometry();
541
542     model = modelaux;
543     model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.75f));
544     model = glm::rotate(model, glm::radians(mainWindow.getllantal()), glm::vec3(0.0f, 0.0f, 1.0f));
545     modelaux = model;
546     model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
547     model = glm::scale(model, glm::vec3(1.25f, 1.0f, 1.25f));
548     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
549     color = glm::vec3(0.3f, 0.3f, 0.3f);
550     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
551     meshList[2]->RenderMeshGeometry();
552     model = modelaux;
553     model = glm::scale(model, glm::vec3(1.2f, 1.2f, 0.45f));
554     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
555     color = glm::vec3(0.3f, 0.3f, 0.3f);
556     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
557     sp.render();
558     model = modelaux;
559     model = glm::scale(model, glm::vec3(2.2f, 0.3f, 1.1f));
560     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
561     color = glm::vec3(0.8f, 0.8f, 0.8f);
562     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
563     meshList[0]->RenderMesh();

```

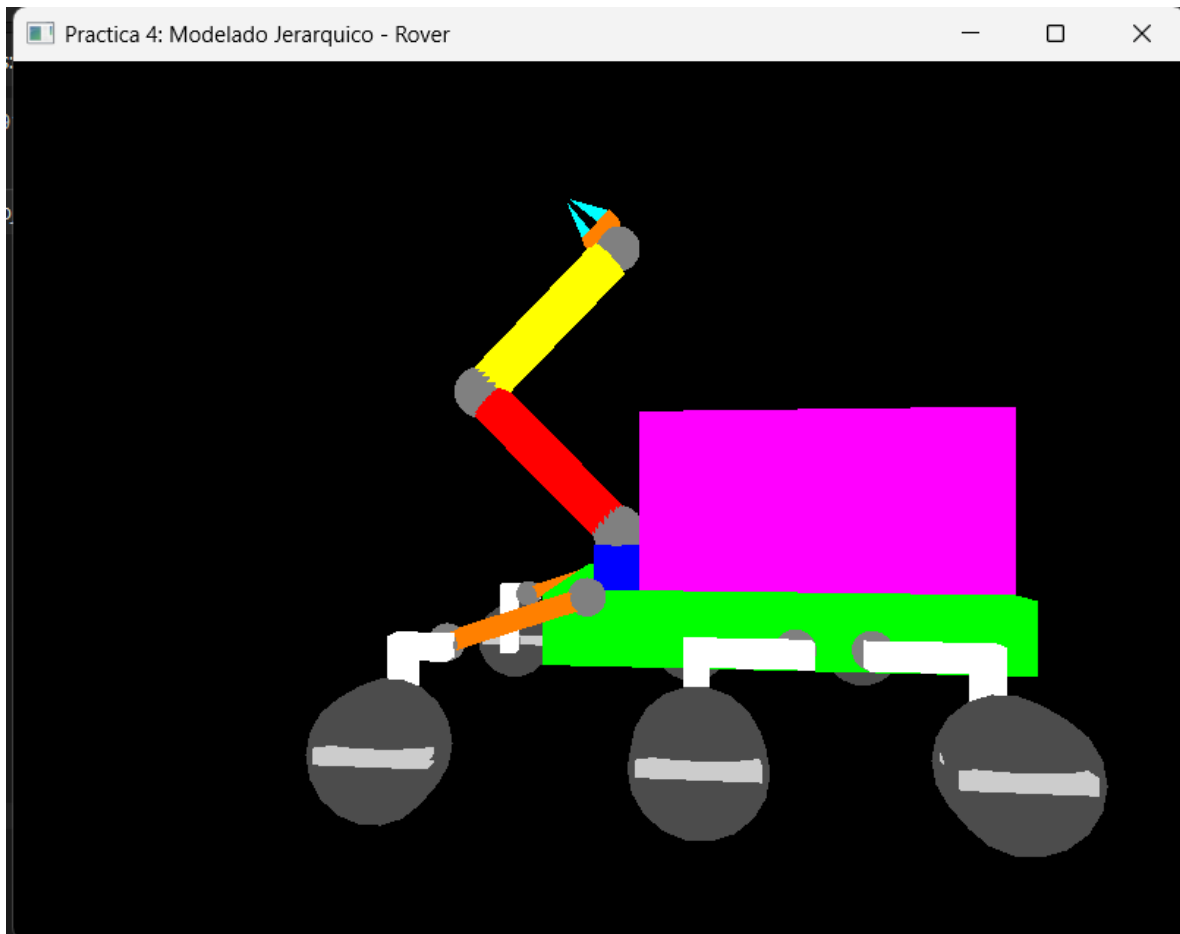
Texto: Cada tecla suma 10 grados a su articulación o llanta.

```

186
187 //BRAZO
188 if (key == GLFW_KEY_F)
189 {
190     theWindow->articulacion1 += 10.0f * sentido; //v
191 }
192 if (key == GLFW_KEY_G)
193 {
194     theWindow->articulacion2 += 10.0f * sentido; //v
195 }
196 if (key == GLFW_KEY_H)
197 {
198     theWindow->articulacion3 += 10.0f * sentido; //v
199 }
200 if (key == GLFW_KEY_J)
201 {
202     //J abre la pinza y Shift + J la cierra; se lim
203     theWindow->articulacion4 += 5.0f * sentido;
204     if (theWindow->articulacion4 > 45.0f)
205     {
206         theWindow->articulacion4 = 45.0f;
207     }
208     if (theWindow->articulacion4 < 0.0f)
209     {
210         theWindow->articulacion4 = 0.0f;
211     }
212 }
213 if (key == GLFW_KEY_K)
214 {
215     theWindow->articulacion5 += 10.0f * sentido; //v
216 }
217 if (key == GLFW_KEY_L)
218 {
219     theWindow->articulacion6 += 10.0f * sentido; //v
220 }

```

```
221
222 //LLANTAS (cada una por separado). Sin Shi
223 if (key == GLFW_KEY_1)
224 {
225     theWindow->llanta1 += 10.0f * sentido;
226 }
227 if (key == GLFW_KEY_2)
228 {
229     theWindow->llanta2 += 10.0f * sentido;
230 }
231 if (key == GLFW_KEY_3)
232 {
233     theWindow->llanta3 += 10.0f * sentido;
234 }
235 if (key == GLFW_KEY_4)
236 {
237     theWindow->llanta4 += 10.0f * sentido;
238 }
239 if (key == GLFW_KEY_5)
240 {
241     theWindow->llanta5 += 10.0f * sentido;
242 }
243 if (key == GLFW_KEY_6)
244 {
245     theWindow->llanta6 += 10.0f * sentido;
246 }
247 }
```



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Me costó colocar las piezas en su lugar correcto. Lo resolví tomando la imagen guía como cuadrícula (cada cuadro = 1 unidad) y calculando la distancia de cada pieza al centro de la base. Aparecieron errores syntax error: identifier 'position' y errores en Camera. La causa era que en el constructor de `_VertexColor` se había borrado el tipo `glm::vec3` antes de `position`. Al corregir esa línea compiló. (Aquí va la captura del error si la tienes.) Al principio no entendía por qué algunas rotaciones movían también a las piezas siguientes. Entendí que si el `rotate` va antes de guardar `modelaux`, lo heredan los hijos. Si va después, solo afecta a esa pieza.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Lo más difícil fue organizar la jerarquía: cada pieza depende de la anterior, y un error en una matriz mueve todo lo que cuelga de ella. El uso de `modelaux` y `modelaux2` permitió que las piezas quedaran unidas y se movieran juntas al girar una articulación.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Me confundió mucho lo de los cuadritos en clase y las posiciones de las piezas, un documento explicando eso hubiera estado muy bien.

- c. Conclusión.

Con esta práctica entendí el modelado jerárquico: cómo se heredan las transformaciones geométricas de una pieza padre a sus hijas mediante matrices auxiliares. Esto permite crear modelos articulados como el rover, donde cada parte se mueve de forma independiente sin separarse del resto.

1. Bibliografía en formato APA

GLFW. (s.f.). Input guide. https://www.glfw.org/docs/latest/input_guide.html

G-Truc Creation. (s.f.). OpenGL Mathematics (GLM). <https://github.com/g-truc/glm>